

Cybersecurity Basics Master Roadmap

A simple, index-wise foundation plan from zero to serious beginner: networking, OS, web, coding, security, labs, and learning order.

Built for learning like a story, not like a dead textbook. Start here, then we can turn each chapter into full lessons with examples, labs, and why/how explanations.

1.0	Roadmap and index only	Foundation to pre-advanced	Follow before pentesting, bug bounty, SOC, cloud security, or malware basics

How to use this roadmap

This PDF is not the full textbook yet. It is the complete index and learning sequence. Think of it like the city map before entering the cyber jungle. If you skip the map, every tool feels magical, every error feels personal, and every hacker tutorial becomes a horror movie with Wireshark subtitles.

For every topic, learn five things: what it is, why it exists, how it works, where it breaks, and how to practice it safely in a lab. That is the pattern we will use later when expanding this roadmap into chapters.

0	Mindset + legality	Know what is allowed, what is illegal, how to use labs and permission properly.	2-3 days
1	Computer + networking basics	Explain how data moves from your app to another computer and back.	3-4 weeks
2	OS + command line	Use Linux and Windows like a technician, not just a user.	3-4 weeks
3	Web + coding	Understand websites, APIs, databases, scripts, and common bugs.	5-7 weeks
4	Security foundations	Know CIA triad, identity, crypto, hardening, logging, threat modeling.	4-6 weeks
5	Practical cyber paths	Choose offensive, defensive, cloud, appsec, malware/reversing, or forensics.	Ongoing

The story: why basics matter

Cybersecurity is not "install Kali, run tool, become ghost in the machine." Real cybersecurity starts with boring things that attackers love: forgotten servers, weak passwords, old software, exposed ports, bad logs, bad permissions, and developers trusting user input like it is their loyal childhood friend. Spoiler: user input is often a demon wearing a text box.

HTTP/1.1 is a good example. HTTP/2 and HTTP/3 are faster and smarter, but HTTP/1.1 still matters because the web is built on compatibility. Proxies, old services, embedded devices, misconfigured servers, debugging tools, and simple APIs still speak it. A security person cannot say "newer exists, so old is useless." Attackers do not care what is fashionable; they care what is exposed.

So this roadmap starts from the stack: electricity becomes bits, bits become packets, packets become protocols, protocols become apps, apps become business, and business becomes attack surface. That is the real cyber story.

Master index: the complete topic map

00	Cyber mindset, ethics, law	Permission, scope, reporting, lab safety, responsible disclosure	None
01	Computer fundamentals	CPU, RAM, storage, processes, files, binary, encoding	None
02	Networking fundamentals	OSI/TCP-IP, IP, subnetting, ports, routing, NAT, firewalls	01
03	Internet services	DNS, DHCP, ARP, ICMP, TCP/UDP, TLS, email basics	02
04	Web foundations	HTTP/1.1, HTTP/2, HTTP/3 idea, browsers, cookies, sessions	02-03
05	Linux basics	Shell, files, users, groups, permissions, services, logs	01
06	Windows basics	NTFS, users, services, registry, event logs, PowerShell	01
07	Programming basics	Python, Bash, PowerShell, JavaScript, C, SQL foundations	01,05,06
08	Databases	Relational models, SQL queries, auth, backups, injection basics	07
09	Virtualization and labs	VMs, snapshots, NAT/bridged networks, safe practice setup	02,05,06

10	Security principles	CIA, AAA, least privilege, zero trust, defense in depth	00-06
11	Cryptography basics	Hashing, encryption, keys, TLS, password storage, PKI	03,04,07
12	Identity and access	AuthN, AuthZ, MFA, sessions, OAuth/OIDC, SSO basics	04,10,11
13	Web security	OWASP Top 10, input validation, XSS, SQLi, SSRF, CSRF, IDOR	04,07,08,12
14	API security	REST, JSON, tokens, rate limits, BOLA, API auth mistakes	04,12,13
15	Wireless basics	Wi-Fi standards, WPA, channels, capture concepts, legal limits	02,03
16	Active Directory basics	Domains, users, groups, Kerberos idea, GPOs, Windows enterprise	06,12
17	Cloud basics	IaaS/PaaS/SaaS, IAM, storage, shared responsibility	02,10,12
18	Containers and DevOps	Docker, images, Kubernetes idea, CI/CD, secrets, supply chain	05,07,17
19	Security operations	Logs, SIEM idea, alerts, detection, vulnerability management	10,13,17
20	Incident response and forensics	Triage, evidence, timelines, disk/memory/network artifacts	05,06,19
21	Malware and reversing basics	Malware types, static/dynamic analysis, assembly idea, safe labs	01,05,06,07
22	Pentesting methodology	Recon, enumeration, exploitation concepts, reporting, cleanup	00-19
23	Bug bounty prerequisites	Scope, impact, duplicates, proof of concept, report quality	00,13,14,22
24	Career and practice plan	Labs, projects, notes, portfolio, certifications, specialization	All

Detailed module roadmap

00. Ethics, legality, and hacker mindset

Why it matters: Cybersecurity without permission is not skill; it is evidence collection for your own funeral. Learn scope, authorization, disclosure, and how to write reports safely.

Core topics: CIA triad, responsible disclosure, bug bounty scope, rules of engagement, data handling, privacy, Indian cyber law awareness, safe lab boundaries.

Practice: Write a fake pentest authorization letter, define in-scope/out-of-scope assets, and create a report template before touching tools.

Real-world example: A researcher finding a real bug but testing beyond scope can turn a valid discovery into a legal problem. Permission is the shield.

01. Computer fundamentals

Why it matters: Attackers abuse how computers actually store, execute, and trust data. You need to know what a process, file, memory, and permission actually mean.

Core topics: Bits/bytes, ASCII/Unicode, binary/hex, CPU, RAM, disk, filesystem, processes, threads, environment variables, PATH, archives, compression.

Practice: Convert text to hex, inspect file metadata, list running processes, compare file hashes, and understand why two same-looking files can be different.

Real-world example: Malware may hide as a normal-looking file name while its extension, hash, or behavior reveals the truth.

02. Networking fundamentals

Why it matters: Cybersecurity is often just understanding who talks to whom, on which port, using which protocol, and whether they should be allowed to.

Core topics: OSI model, TCP/IP model, MAC, IP, subnetting, CIDR, ports, TCP handshake, UDP, routing, NAT, firewall basics, VLAN idea.

Practice: Use ping, ip, ifconfig, traceroute, netstat/ss, tcpdump/Wireshark. Draw your home network and label IPs, gateway, DNS, NAT.

Real-world example: An exposed database port on the internet is not a "database problem" only; it is a network exposure problem first.

03. Internet protocols and services

Why it matters: DNS, DHCP, ARP, ICMP, TLS, and email are the plumbing of the internet. When plumbing leaks, attackers bring buckets.

Core topics: DNS records, recursive vs authoritative DNS, DHCP leases, ARP, ICMP, TCP/UDP services, TLS certificates, SMTP/IMAP/POP basics.

Practice: Run dig/nslookup, inspect DNS records, capture a DNS lookup, view a TLS certificate, understand why HTTPS lock does not mean the website is safe.

Real-world example: Phishing domains use DNS and TLS too. A green lock means encrypted connection, not honest business.

04. Web foundations

Why it matters: Most modern attacks touch the web somewhere: login pages, APIs, dashboards, cloud consoles, admin panels, or forgotten staging sites.

Core topics: HTTP methods, status codes, headers, cookies, sessions, cache, CORS idea, browser same-origin policy, HTML/CSS/JS, HTTP versions.

Practice: Open browser DevTools, inspect requests, modify headers in a local lab, compare GET vs POST, trace login cookie behavior.

Real-world example: An app can use modern HTTP/2 but still fail because it trusts a user ID in a URL. Speed does not fix broken authorization.

05. Linux basics

Why it matters: Servers, cloud, containers, security tools, and many labs run on Linux. If Linux scares you, Kali becomes expensive wallpaper.

Core topics: Shell navigation, permissions, sudo, users/groups, services, systemd, logs, packages, cron, SSH, processes, networking commands.

Practice: Complete OverTheWire Bandit basics, manage files, use grep/find/sed/awk lightly, read logs, start/stop services.

Real-world example: A bad file permission can leak secrets. A readable .env file can expose database passwords faster than a movie hacker monologue.

06. Windows basics

Why it matters: Most companies run Windows endpoints and Active Directory. Defensive and offensive work both require Windows fluency.

Core topics: Users/groups, NTFS permissions, services, registry, event viewer, Windows Defender, PowerShell, SMB, RDP, scheduled tasks.

Practice: Create local users, inspect event logs, learn PowerShell basics, understand UAC, list services, map network shares in a lab.

Real-world example: WannaCry spread through Windows SMB exposure and unpatched systems. Basic patching and service exposure matter.

07. Programming and scripting

Why it matters: Coding lets you automate boring work, understand vulnerabilities, read exploit logic, build tools, and stop worshipping copy-paste gods.

Core topics: Python fundamentals, Bash scripting, PowerShell, JavaScript, SQL, C basics, data structures basics, regex, JSON, HTTP requests.

Practice: Write port checkers, log parsers, wordlist cleaners, HTTP clients, CSV processors, and small automation scripts.

Real-world example: A pentester who can script can test 1,000 endpoints carefully; a copy-paster can break one endpoint loudly.

08. Databases and SQL

Why it matters: Apps store valuable data in databases. Many major breaches are not about "hacking the database" but tricking apps into asking bad questions.

Core topics: Tables, rows, keys, joins, indexes, SQL queries, users/roles, backups, connection strings, ORM idea, SQL injection concept.

Practice: Create a small SQLite/PostgreSQL database, query it, parameterize inputs in code, and compare safe vs unsafe queries in a local toy app.

Real-world example: SQL injection happens when user input becomes part of a command instead of data. The database obeys exactly the wrong thing.

09. Virtualization and lab building

Why it matters: You need a safe place to break things. Production is not your playground; it is someone else's paycheck.

Core topics: VirtualBox/VMware, snapshots, NAT vs bridged networking, host-only networks, vulnerable VMs, Docker labs, isolation.

Practice: Build Kali + Ubuntu + Windows lab, take snapshots, test Nmap only inside your lab, document IP ranges.

Real-world example: A snapshot lets you reverse your disasters. Without snapshots, your lab becomes archaeology.

10. Security principles

Why it matters: Tools change, principles survive. Least privilege, patching, logging, segmentation, and threat modeling are everywhere.

Core topics: CIA triad, AAA, least privilege, defense in depth, secure defaults, attack surface, risk, threat, vulnerability, exploit, impact.

Practice: Pick any app and list its assets, users, trust boundaries, threats, and controls.

Real-world example: Equifax became a famous case study in basics: asset awareness, patching, and response discipline.

11. Cryptography basics

Why it matters: Crypto protects secrets, but wrong crypto creates expensive decoration. You need concepts, not custom encryption delusions.

Core topics: Hashing, salting, password storage, symmetric/asymmetric encryption, digital signatures, TLS, certificates, key management, randomness.

Practice: Hash files, compare SHA-256 outputs, generate keys, inspect TLS certs, learn why passwords need slow hashing.

Real-world example: Storing passwords with plain SHA-256 is weak because attackers can guess fast. Password hashing must slow attackers down.

12. Identity and access

Why it matters: Modern breaches often start with identity: stolen password, weak MFA, leaked token, overprivileged account.

Core topics: Authentication, authorization, sessions, cookies, JWT basics, MFA, password reset, OAuth/OIDC overview, SSO, service accounts.

Practice: Trace a login flow in DevTools, decode a JWT safely, compare session cookie vs access token, design a secure password reset flow.

Real-world example: An admin token leaked in logs can be worse than a password leak because tools trust tokens silently.

13. Web application security

Why it matters: Web bugs are where theory becomes real: one bad check, one trusted parameter, one unsafe query, one exposed admin route.

Core topics: OWASP Top 10, access control, injection, XSS, CSRF, SSRF, file upload bugs, deserialization idea, security misconfiguration.

Practice: Use PortSwigger Web Security Academy, OWASP Juice Shop, DVWA, and OverTheWire Natas. Always stay in labs or authorized scopes.

Real-world example: IDOR: changing /user/100 to /user/101 should not show another user's data, but weak apps sometimes trust the URL.

14. API security

Why it matters: Mobile apps, dashboards, AI products, and modern websites speak through APIs. APIs often leak logic behind pretty UI.

Core topics: REST, JSON, status codes, API keys, bearer tokens, rate limits, BOLA/IDOR, mass assignment, pagination, webhooks.

Practice: Use Postman/Bruno/curl against a local API. Test authorization logic, not random violence against endpoints.

Real-world example: A mobile app may hide a button, but the API still exists. Security must live on the server, not in the UI costume.

15. Wireless basics

Why it matters: Wireless is networking through air. That means signals, channels, encryption, clients, access points, and legal boundaries.

Core topics: 802.11 basics, SSID/BSSID, channels, WPA2/WPA3, handshake concept, evil twin concept, captive portals, Bluetooth basics.

Practice: Use Wireshark monitor mode only in your own lab. Learn what frames look like. Do not capture public Wi-Fi users' traffic.

Real-world example: Knowing a Wi-Fi password does not give permission to inspect other people's private traffic. That is how curiosity becomes a court date.

16. Active Directory basics

Why it matters: AD is the kingdom map in many companies. Users, computers, groups, policies, and authentication all meet there.

Core topics: Domain, forest, OU, users, groups, GPO, Kerberos idea, LDAP idea, domain controllers, privilege paths.

Practice: Build a tiny Windows Server lab later, create users/groups, apply GPO, observe login events.

Real-world example: One overprivileged helpdesk account can become a stepping stone to domain compromise if controls are weak.

17. Cloud security basics

Why it matters: Cloud does not remove security responsibility; it rearranges it. Misconfigured storage and IAM are common villains.

Core topics: IaaS/PaaS/SaaS, AWS/Azure/GCP basics, IAM, buckets/storage, security groups, keys, logs, shared responsibility.

Practice: Use free-tier carefully, create least-privilege IAM user, enable logs, make a private bucket, understand public exposure.

Real-world example: A public storage bucket can leak data without any exploit. Misconfiguration is the exploit wearing a business suit.

18. Containers, DevOps, and supply chain

Why it matters: Modern apps ship through pipelines, images, dependencies, and secrets. Attackers love the factory, not just the final product.

Core topics: Docker images/containers, Dockerfile, registries, Kubernetes idea, CI/CD, secrets, dependency scanning, SBOM idea.

Practice: Containerize a toy app, scan dependencies, avoid hardcoded secrets, learn .gitignore, run minimal containers.

Real-world example: A secret pushed to GitHub can be stolen quickly. Git history remembers your sins like an elephant with broadband.

19. Defensive security and security operations

Why it matters: Defense is about visibility and response. If you cannot see what happened, you are not secure; you are just optimistic.

Core topics: Logs, SIEM, detection rules, EDR idea, vulnerability management, patching, baselines, alert triage, threat intel basics.

Practice: Collect Linux auth logs, Windows event logs, write simple Sigma-style detection ideas, build a vulnerability tracker spreadsheet.

Real-world example: A failed-login storm followed by one success is not random noise. It may be password spraying with a tiny victory dance.

20. Incident response and digital forensics

Why it matters: When something breaks, panic deletes evidence. IR teaches calm, containment, evidence, timeline, recovery, and lessons learned.

Core topics: Preparation, detection, containment, eradication, recovery, lessons learned, chain of custody, disk/memory/network artifacts.

Practice: Write an incident checklist. Build timelines from logs. Practice preserving evidence in a lab before touching suspicious files.

Real-world example: Deleting malware immediately may destroy clues about how it entered. Sometimes the first job is observe, not smash.

21. Malware and reverse engineering basics

Why it matters: You do not need to be a wizard immediately, but you should know how malware behaves and how analysts study it safely.

Core topics: Malware types, persistence, C2 idea, static vs dynamic analysis, hashes, strings, sandboxing, assembly basics, PE/ELF idea.

Practice: Analyze harmless sample programs, use strings/file/hash tools, never run unknown malware on your host OS.

Real-world example: A suspicious binary calling strange domains may reveal command-and-control behavior without needing full reverse engineering.

22. Penetration testing methodology

Why it matters: Pentesting is not tool spam. It is scoped investigation, evidence collection, controlled testing, and clear reporting.

Core topics: Scoping, recon, enumeration, vulnerability validation, exploitation in lab/authorized scope, privilege escalation concept, reporting, remediation.

Practice: Run a full lab assessment on a vulnerable VM: notes, screenshots, risk rating, remediation, executive summary.

Real-world example: Finding open port 22 is not a finding by itself. Finding weak SSH credentials in scope with proof and impact is a finding.

23. Bug bounty prerequisites

Why it matters: Bug bounty rewards impact, clarity, and scope discipline. Random scanning and duplicate low-quality reports get you sadness speedrun.

Core topics: Program scope, safe testing, duplicate checking, severity, proof of concept, business impact, report writing, communication.

Practice: Write practice reports for lab bugs. Include steps, evidence, impact, remediation, and what you did not test.

Real-world example: "XSS exists" is weaker than "stored XSS lets attacker act as victim on admin dashboard." Impact is the money language.

24. Career path and projects

Why it matters: Learning becomes real when you build receipts: notes, labs, tools, reports, portfolio, and consistent practice.

Core topics: Choose path: pentest, SOC, appsec, cloud security, DFIR, malware. Build projects and document them professionally.

Practice: Create a GitHub repo for notes/tools, a portfolio page, 3 lab reports, 2 Python tools, 1 security checklist, 1 write-up series.

Real-world example: Recruiters do not read your soul. They read proof: projects, reports, clarity, and consistency.

Recommended learning order: 90-day foundation plan

1-7	Ethics, computer basics, file/process concepts	Notes explaining process, file, hash, permission	Linux terminal, Windows Task Manager/Event Viewer
8-21	Networking: IP, subnet, ports, TCP/UDP, routing, DNS	Draw your network and explain packet journey	Wireshark, ping, traceroute, dig, ss/netstat
22-35	Linux + Windows command line	Command cheat sheet and log-reading notes	OverTheWire Bandit, PowerShell basics
36-50	Python, Bash, JavaScript, SQL basics	3 scripts: log parser, URL checker, wordlist cleaner	Python, Bash, SQLite/Postgres
51-65	Web, HTTP, cookies, sessions, auth basics	Explain one login flow with screenshots	Browser DevTools, curl, Burp Community in labs
66-78	OWASP Top 10 and API security basics	3 lab writeups: IDOR, XSS, SQLi concept	Juice Shop, PortSwigger Academy, Natas
79-90	Defensive basics, cloud/container basics, reporting	One mini pentest-style lab report	Nmap in lab, Docker, logs, report template

Minimum tools to learn safely

Wireshark	See packets and protocols instead of guessing	Capture your own lab traffic
Nmap	Discover hosts, ports, and services	Scan only your lab machines or authorized targets
Burp Suite Community	Understand web requests and responses	Use with local labs and legal training sites
curl/httpie	Make HTTP requests from terminal	Test your own APIs and labs
Python	Automation and security scripting	Write parsers/checkers, not malware
Git/GitHub	Track projects and notes	Publish safe scripts and writeups
Docker	Run isolated learning apps	Run Juice Shop/local apps safely
VirtualBox/VMware	Create labs and snapshots	Practice without touching real systems

What not to skip

These are the boring monsters beginners skip, then later cry over:

- Subnetting and ports - because every scan result depends on them.
- HTTP and cookies - because most web security starts here.
- Linux permissions and logs - because servers expose their story there.
- Windows basics and Active Directory - because companies run on them.
- SQL and databases - because data is the treasure chest.
- Authentication vs authorization - because many serious bugs are auth logic failures.
- Reporting - because a bug without clear impact is just a spooky screenshot.
- Legal scope - because prison has terrible Wi-Fi.

Safe practice sources

Use legal labs and intentionally vulnerable systems. Do not practice on random public websites, public Wi-Fi users, college systems, or company assets unless you have explicit written permission.

Linux basics	OverTheWire Bandit	Terminal, files, SSH, permissions
Web basics	OverTheWire Natas, Juice Shop, PortSwigger Academy	HTTP, auth, XSS, SQLi, access control
Networking	Your own VMs and home lab	Packet capture, ports, DNS, routing
Defensive security	Local logs from Linux/Windows VMs	Detection thinking and timelines
Pentest reporting	Vulnerable VM in isolated lab	Professional notes and remediation

Final roadmap checklist

- Can I explain how a browser reaches a website from DNS to TLS to HTTP?
- Can I read a basic Wireshark capture and identify DNS, TCP, TLS, HTTP?
- Can I use Linux terminal confidently for files, permissions, processes, services, logs?
- Can I use Windows Event Viewer and PowerShell basics?
- Can I write small Python/Bash scripts to automate boring tasks?
- Can I explain authentication, authorization, sessions, cookies, tokens, and MFA?
- Can I explain SQL injection, XSS, CSRF, SSRF, IDOR/BOLA in simple language?
- Can I build and reset a safe lab with VMs/snapshots?
- Can I write a clean vulnerability report with impact and remediation?
- Can I say no to illegal testing even when curiosity is screaming like a haunted router?

Source base for future detailed chapters

When we expand this roadmap into the full storytelling book, use primary and trusted sources: RFC Editor/IETF for internet standards; MDN for web concepts; NIST CSF and SP 800-series for security frameworks, identity, patching, and incident response; OWASP for web/app security; MITRE ATT&CK/D3FEND for attacker and defender models; Microsoft Learn for Windows/PowerShell/Active Directory; Linux kernel/GNU docs for Linux and Bash; Docker/Kubernetes/AWS docs for modern infrastructure; Wireshark/Nmap official docs for tooling.

Important note: this PDF is an index and learning roadmap. The next step is to choose Module 01 and expand it into a full chapter with story, examples, diagrams, labs, commands, mistakes, and Q&A.;